

Visão Geral

O fluxo do teste inicia no endpoint **/webhooks/{gateway}**, responsável por receber eventos dos gateways Lous e Grummer. Todo payload recebido é persistido na tabela **raw_payloads**, garantindo auditoria completa mesmo em cenários de falha. Após a persistência inicial, o sistema realiza a descryptografia dos payloads do gateway Grummer utilizando AES-256-CBC com PKCS7. Em seguida ocorre a validação estrutural através do Pydantic, normalização dos dados de entrada (nome, telefone e e-mail) e verificação de idempotência baseada na chave natural (**transaction_id, event**).

Somente eventos **order.approved** com pagamento aprovado seguem para processamento assíncrono através da fila **lead.received**.

O consumer principal realiza a persistência transacional das entidades de negócio (**leads, orders, lead_events e distribution_status**) utilizando Stored Procedure para garantir consistência. Ou tudo salva ou nada salva! Após a persistência, são publicados eventos para os canais de distribuição. O canal SMS foi implementado conforme solicitado no desafio, consumindo a fila **dist.sms**, enviando mensagens para um endpoint HTTP externo e atualizando o status de distribuição. Em caso de falha, o worker executa retry exponencial (1s, 4s e 16s). Após a terceira tentativa sem sucesso, a mensagem é enviada para a DLQ **dist.dead.sms** e registrada na tabela **lead_dead_letter**.

A solução também implementa observabilidade através de logs estruturados, Correlation ID e métricas Prometheus.

Decisões Arquiteturais

Persistência antes do processamento

A persistência do payload bruto ocorre antes de qualquer validação ou transformação. Essa decisão garante auditoria completa dos eventos recebidos, rastreabilidade de erros e recuperação de informações mesmo em falhas de decrypt ou schema.

Processamento assíncrono

RabbitMQ foi utilizado para desacoplar o recebimento do webhook do processamento de negócio. Os benefícios são menor tempo de resposta da API, escalabilidade independente entre produtores e consumidores e maior tolerância a picos de carga.

Idempotência

Foi adotada a chave natural (**transaction_id, event**). Essa abordagem impede reprocessamentos causados por reenvios de webhooks sem bloquear eventos legítimos de um mesmo pedido, como approved, refunded ou canceled.

Retry com Backoff Exponencial

Implementado para evitar pressão excessiva sobre serviços externos. Sequência de [1 segundo, 4 segundos e 16 segundos]. Após esgotar as tentativas, a mensagem é encaminhada para DLQ.

Pontos Importantes

Apenas eventos **order.approved** vão para esteira de distribuição. Eventos não aprovados são armazenados apenas para auditoria. Apenas o canal SMS foi implementado, conforme solicitado. Os canais Email, WhatsApp e Call Center foram modelados para demonstrar a arquitetura proposta. O banco MySQL foi utilizado como persistência principal. O processamento assume que a chave (**transaction_id, event**) identifica unicamente um evento.

Justificativa dos Índices

Os índices foram criados considerando as consultas de auditoria exigidas pelo desafio, e os outputs das queries de auditoria e seus respectivos planos de execução **EXPLAIN** estão disponíveis em **evidence_database/audit_queries/** com um README explicativo também.

lead_events

Índice (**event, persisted_at**). O objetivo foi acelerar consultas de reconciliação e melhorar filtros por período.

distribution_status

Índices: (**status**) / (**channel, status**). O objetivo foi consultas de mensagens pendentes e taxa de sucesso por canal.

lead_dead_letter

Índice (**souce, created_at**). O objetivo foi auditoria de erros e identificação rápida de falhas por origem.

Os índices foram validados utilizando **EXPLAIN**, reduzindo full scans e melhorando a eficiência das consultas de auditoria. Apenas tables com pouca persistência que o próprio otimizador do sql opta por fazer full table scan.

Escolha da Linguagem e Bibliotecas

Python -> Escolhido pela produtividade, legibilidade e forte ecossistema para integração, mensageria e APIs.

FastAPI -> Utilizado devido a alto desempenho assíncrono, integração nativa com Pydantic, documentação automática e facilidade de validação de contratos.

RabbitMQ -> Escolhido por oferecer filas duráveis, modelo robusto de mensageria facilidade para implementar retries e DLQs.

MySQL -> Utilizado por sua maturidade, simplicidade operacional e suporte adequado ao volume do desafio.

aio-pika e aiohttp -> Bibliotecas assíncronas utilizadas para comunicação com RabbitMQ e integrações HTTP, permitindo melhor aproveitamento do modelo assíncrono do FastAPI.